

Aqui está a documentação técnica completa do projeto **SIG-Apple** formatada em Markdown detalhado. Você pode copiá-la diretamente para instruir outro modelo/chat a dar continuidade ao desenvolvimento de todo o sistema.

Para salvar como **PDF**, basta copiar o texto abaixo, colar no [Dillinger.io](https://dillinger.io) ou no próprio VS Code e usar a opção **Imprimir / Salvar como PDF** do navegador.

Documentação de Especificação Técnica — SIG-Apple

1. Visão Geral do Projeto

O **SIG-Apple** é um sistema de gestão integrada especializado em assistência técnica e manutenção de smartphones (foco em dispositivos Apple/iPhones). O sistema gerencia o cadastro de clientes, dispositivos, checklist de entrada/saída, precificação dinâmica baseada em regras de peças/mão de obra e abertura de Ordens de Serviço (OS).

2. Stack Tecnológica

Architecture Pattern

- **Monorepo (Single Repository):** Gerenciado via scripts unificados com `concurrently` na raiz.

Backend

- **Runtime:** Node.js (v24+)
- **Linguagem:** TypeScript
- **Framework:** Express.js
- **Driver de Banco de Dados:** `mysql2/promise`
- **Utilitários:** `dotenv`, `cors`, `uuid`
- **Execução em Desenvolvimento:** `tsx watch`

Frontend

- **Build Tool & Dev Server:** Vite (v8+)
- **Linguagem:** TypeScript
- **Biblioteca UI:** React (v19+)

Banco de Dados

- **SGBD:** MySQL (Servidor Local na porta `3306`)
- **Charset/Collation:** `utf8mb4` / `utf8mb4_unicode_ci`

3. Estrutura de Arquivos e Diretórios

```
Davi.Tec/  
├─ package.json          # Configuração da raiz (Script de start  
concorrente)  
├─ backend/  
│   ├── .env             # Variáveis de ambiente (PORT=3000, DATABASE_URL)  
│   └─ package.json      # Dependências do servidor (express, mysql2, uuid,  
etc)  
│   ├── tsconfig.json    # Configuração do TypeScript no Backend  
│   └─ src/  
│       └─ server.ts      # API REST, conexão com pool MySQL e regras de  
negócio  
└─ frontend/  
    ├── package.json      # Dependências da interface (vite, react, react-  
dom)  
    ├── vite.config.ts    # Configurações da build do Vite  
    └─ src/  
        └─ App.tsx        # Interface do usuário (Formulários, Checklist,  
Integração API)
```

4. Configuração de Scripts e Raiz (`package.json`)

```
{  
  "name": "davi.tec",  
  "version": "1.0.0",  
  "description": "Monorepo Davi.Tec - Frontend e Backend SIG-Apple",  
  "private": true,  
  "scripts": {  
    "dev:backend": "npm run dev --prefix backend",  
    "dev:frontend": "npm run dev --prefix frontend",  
    "dev": "concurrently \"npm run dev:backend\" \"npm run dev:frontend\"",  
  },  
  "devDependencies": {  
    "concurrently": "^10.0.5"  
  }  
}
```

5. Schema do Banco de Dados (MySQL)

```
CREATE DATABASE IF NOT EXISTS sig_apple_db  
DEFAULT CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;  
  
USE sig_apple_db;
```

```
-- 1. Tabela de Clientes
CREATE TABLE IF NOT EXISTS clientes (
  id_cliente VARCHAR(36) PRIMARY KEY,
  nome VARCHAR(150) NOT NULL,
  cpf_cnpj VARCHAR(18) UNIQUE NOT NULL,
  whatsapp VARCHAR(20) NOT NULL,
  email VARCHAR(100),
  data_cadastro DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 2. Tabela de Aparelhos por Cliente
CREATE TABLE IF NOT EXISTS aparelhos_cliente (
  id_aparelho VARCHAR(36) PRIMARY KEY,
  id_cliente VARCHAR(36) NOT NULL,
  modelo VARCHAR(100) NOT NULL,
  imei_1 VARCHAR(18),
  senha_desbloqueio VARCHAR(100),
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE CASCADE
);

-- 3. Regras de Precificação Dinâmica
CREATE TABLE IF NOT EXISTS regras_precificacao (
  id_regra VARCHAR(36) PRIMARY KEY,
  nome_servico VARCHAR(150) NOT NULL,
  categoria VARCHAR(50) NOT NULL, -- 'FORMULA_PECA', 'TAMPA', 'TABELA_FIXA'
  faixa_modelo VARCHAR(100) NOT NULL,
  multiplicador_pecas DECIMAL(4,2) DEFAULT 2.00,
  mao_de_obra_padrao DECIMAL(10,2) DEFAULT 50.00,
  frete_padrao DECIMAL(10,2) DEFAULT 20.00,
  custos_adicionais_padrao DECIMAL(10,2) DEFAULT 20.00,
  preco_fixo_venda DECIMAL(10,2) DEFAULT 0.00,
  ativo BOOLEAN DEFAULT TRUE
);

-- 4. Ordens de Serviço (OS)
CREATE TABLE IF NOT EXISTS ordens_servico (
  id_os VARCHAR(36) PRIMARY KEY,
  numero_os VARCHAR(20) UNIQUE NOT NULL,
  id_cliente VARCHAR(36) NOT NULL,
  id_aparelho VARCHAR(36) NOT NULL,
  status_os VARCHAR(50) NOT NULL DEFAULT 'AGUARDANDO_AVALIACAO',
  defeito_relatado TEXT NOT NULL,
  checklist_entrada JSON,
  subtotal_servicos DECIMAL(10,2) DEFAULT 0.00,
  desconto_valor DECIMAL(10,2) DEFAULT 0.00,
  valor_total DECIMAL(10,2) DEFAULT 0.00,
  lucro_estimado_total DECIMAL(10,2) DEFAULT 0.00,
  data_abertura DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
  FOREIGN KEY (id_aparelho) REFERENCES aparelhos_cliente(id_aparelho)
);

-- Carga inicial de Regras de Precificação
```

```
INSERT INTO regras_precificacao (id_regra, nome_servico, categoria, faixa_modelo,
multiplicador_peca, mao_de_obra_padrao, frete_padrao, custos_adicionais_padrao,
preco_fixo_venda) VALUES
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11', 'Troca de Tela/Bateria/Câmera',
'FORMULA_PECA', 'iPhones 11 ao 13 Pro', 2.00, 50.00, 20.00, 20.00, 0.00),
('b2eebc99-9c0b-4ef8-bb6d-6bb9bd380a22', 'Troca de Tela/Bateria/Câmera',
'FORMULA_PECA', 'iPhones 13 Pro Max ao 14 Pro Max', 2.00, 80.00, 20.00, 20.00,
0.00),
('c3eebc99-9c0b-4ef8-bb6d-6bb9bd380a33', 'Troca de Tela/Bateria/Câmera',
'FORMULA_PECA', 'iPhones 15 em diante', 2.00, 100.00, 20.00, 20.00, 0.00),
('f6eebc99-9c0b-4ef8-bb6d-6bb9bd380a66', 'Reativação de Bateria', 'TABELA_FIXA',
'iPhones até o 13 PM', 1.00, 130.00, 0.00, 0.00, 130.00);
```

6. Endpoints da API REST (Backend Express)

POST /v1/orcamentos/calcular

- **Finalidade:** Realiza o cálculo dinâmico de precificação com base nos custos de peças e regras do banco.
- **Regras de Negócio:**
- Categoria **FORMULA_PECA**: $\text{Subtotal} = (\text{Custo Peça} \times \text{Multiplicador}) + \text{Mão de Obra} + \text{Frete} + \text{Custos}$
- Lucro Real: $\text{Subtotal} - (\text{Custo Peça} + \text{Frete} + \text{Custos Operacionais})$

POST /v1/ordens-servico

- **Finalidade:** Inicia uma transação MySQL para gravar o cliente (caso não exista), cadastrar o aparelho associado e criar a OS com a estrutura JSON de checklist.
- **Retorno:** Retorna o **idOs** (UUID) e o **numeroOs** gerado.

7. Próximos Passos Recomendados para Desenvolvimento

1. Criar módulo de autenticação de técnicos e administradores (JWT).
2. Desenvolver a visualização em grade (Kanban) para alteração de status da OS (**EM_ANALISE**, **AGUARDANDO_PECA**, **PRONTO**, **ENTREGUE**).
3. Gerar modelo de impressão em PDF para Certidão de Garantia e Comprovante de Entrada.
4. Integrar envio automático de notificações via WhatsApp Web/API.